

Arhitectura sistemelor de calcul paralel - examen

23 iunie 2025

Durata examenului este de două ore. Tratați următoarele subiecte (soluția dumneavoastră va fi sub forma unor coduri sursă C MPI).

1. **(4p)** Scrieți un program C MPI cu următoarea funcționalitate:
 - (a) cele $nproc$ (număr par!) procese lansate în paralel sunt separate în două grupuri distincte, cele cu rang par într-un grup (desemnat în cele ce urmează G1), cele cu rang impar în celălalt (desemnat în cele ce urmează G2);
 - (b) procesul cu rang 0 din G1 citește datele din fișierul *input1.dat* într-un bloc alocat dinamic (de ex. *float data*), le distribuie uniform tuturor proceselor din G1 care le stochează în blocuri alocate dinamic (de ex. *float local_data*) și apoi calculează sumele parțiale $\sum_i \frac{x_i^2}{(x_i^4+1)} \exp(-x_i^2)$ incluzând toate elementele x_i din *float local_data*;
 - (c) rezultatul este colectat cu reducere (sumare) de la toate procesele din G1 de către procesul cu rang 0, care scrie fișierul *output1.dat* cu următorul conținut
N:[TAB] val [NEWLINE]
Suma ponderată:[TAB] val [NEWLINE]
(N este numărul de valori din fișierul *input1.dat*, valoarea de pe linia a doua fiind suma ponderată calculată);
 - (d) procesul cu rang 0 din G2 citește datele din fișierul *input2.dat* într-un bloc alocat dinamic (de ex. *float data*), le distribuie uniform tuturor proceselor din G2 care le stochează în blocuri alocate dinamic (de ex. *float local_data*) și apoi calculează sumele parțiale $\sum_i \frac{x_i^2}{\log(1+x_i^4)} \exp(-x_i^2)$ incluzând toate elementele x_i din *float local_data*;
 - (e) rezultatul este colectat cu reducere (sumare) de la toate procesele din G2 de către procesul cu rang 0, care scrie fișierul *output2.dat* cu următorul conținut:
N:[TAB] val [NEWLINE]
Suma ponderată:[TAB] val [NEWLINE]
(N este numărul de valori din fișierul *input1.dat*, valoarea de pe linia a doua fiind suma ponderată calculată);
 - (f) Funcții MPI sugerate: *MPI_Init()*, *MPI_Comm_size()*, *MPI_Comm_rank()*, *MPI_Comm_group()*, *MPI_Group_incl()*, *MPI_Comm_create()*, *MPI_Group_rank()*, *MPI_Scatter()*, *MPI_Reduce()*, *MPI_Finalize()*.
2. **(5p)** Un *wavelet* este un pachet de unde (Fig. 1) utilizat în procesarea semnalelor. Transformata wavelet continuă a semnalului $f(t)$ este definită de:

$$w(s, \tau) = \int_{-\infty}^{\infty} f(t) \psi_{s,\tau}^*(t) dt, \quad (1)$$

în care $\psi_{s,\tau}(t)$ sunt funcțiile wavelet de bază. Transformata inversă este:

$$f(t) = \int_0^{\infty} ds \int_{-\infty}^{\infty} w(s, \tau) \psi_{s,\tau}(t) d\tau. \quad (2)$$

Varianta discretă utilizată în analiza numerică a semnalelor este construită pe baza setului ortonormat $\{\psi_{m,n}(t) = \frac{1}{\sqrt{a^m}} \psi\left(\frac{t-nb}{a^m}\right)\}$ cu $m, n \in \mathbb{Z}$, $a > 1$, $b > 0$ ($\psi(t)$ se numește *pachetul mamă - mother wavelet*).¹ Codul sursă C *wavelet.c* disponibil alăturat implementează o procedură de filtrare a unui semnal disponibil în fișierul *data.in* folosind primele 20 cele mai mari componente ale transformatei wavelet. Semnalul procesat este scris apoi în fișierul *wavelet.signal.dat*. Vi se cere să paralelizați acest program, astfel încât să funcționeze cu 4 procese, cu următoarea procedură:

¹Textul referitor la transformata wavelet are un caracter pur informativ. Nu este necesar să cunoașteți mai multe detalii despre subiect pentru a rezolva problema propusă.

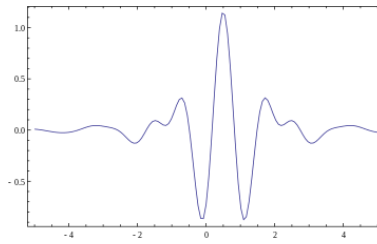


Figura 1: Wavelet - semnal cu amplitudine variabilă, care crește de la zero și scade înapoi la zero, utilizat în procesarea semnalelor.

- (a) toate procesele declară *int nc=5* și suplimentar variabila *int nl* și blocul *double * ldata* cărui i se alocă spațiu în memoria liberă pentru $nl = n/nproc$ valori *double*;
- (b) toate procesele alocă pentru *abscoeff* spațiu în memoria liberă pentru *nl* valori *double*;
- (c) toate procesele alocă pentru *p* spațiu în memoria liberă pentru *nl* valori *size_t*;
- (d) procesul cu rangul 0 este cel care efectuează operațiile I/O (citește fișierul de intrare în *data* pentru care alocă spațiu pentru *n* valori *double*, scrie fișierul de ieșire)
- (e) procesul cu rangul 0 trimite segmente egale din buffer-ul *data* către toate procesele (*MPI_Scatter()* este o bună alegere);
- (f) urmează partea secvențială, în care fiecare proces aplică transformata wavelet directă asupra valorilor locale *ldata* (atenție că acest bloc conține *nl* valori!), filtrează primele $nc = 5$ componente și aplică transformata wavelet inversă, la fel ca în codul secvențial;
- (g) rezultatul este colectat de procesul cu rangul 0 înapoi în blocul *data* (funcția *MPI_Gather()* este o bună alegere), care îl scrie apoi în fișierul de ieșire;
- (h) datele de intrare și de ieșire pot fi vizualizate cu *gnuplot* direct sau cu funcțiile de bibliotecă de interfață cu *gnuplot* create la începutul semestrului;
- (i) Funcții MPI sugerate: *MPI_Init()*, *MPI_Comm_size()*, *MPI_Comm_rank()*, *MPI_Scatter()*, *MPI_Gather()*, *MPI_Finalize()*.

Se acordă 1p din oficiu. Funcțiile MPI indicate mai sus sunt doar sugestii de lucru. Orice soluție corectă, care asigură funcționalitatea cerută, va fi luată în considerare. Alocarea judicioasă a resurselor de calcul necesare va fi avută în vedere la acordarea notei. Întrucât sunt utilizate funcții incluse în biblioteca GSL, compilarea se face cu:

```
mpicc -o nume_program fisier_sursa.c -lgslcblas -lgsl -lm
```

Dacă sistemul dumneavoastră de operare este Linux Ubuntu, compilați cu:

```
mpicc -o nume_program fisier_sursa.c -lgsl -lgslcblas -lm
```

Succes!